



中国科学技术大学
University of Science and Technology of China

计算机系统概论A

Introduction to Computing Systems
(CS1002A.03)

Chapter 9-1 Memory Mapped I/O

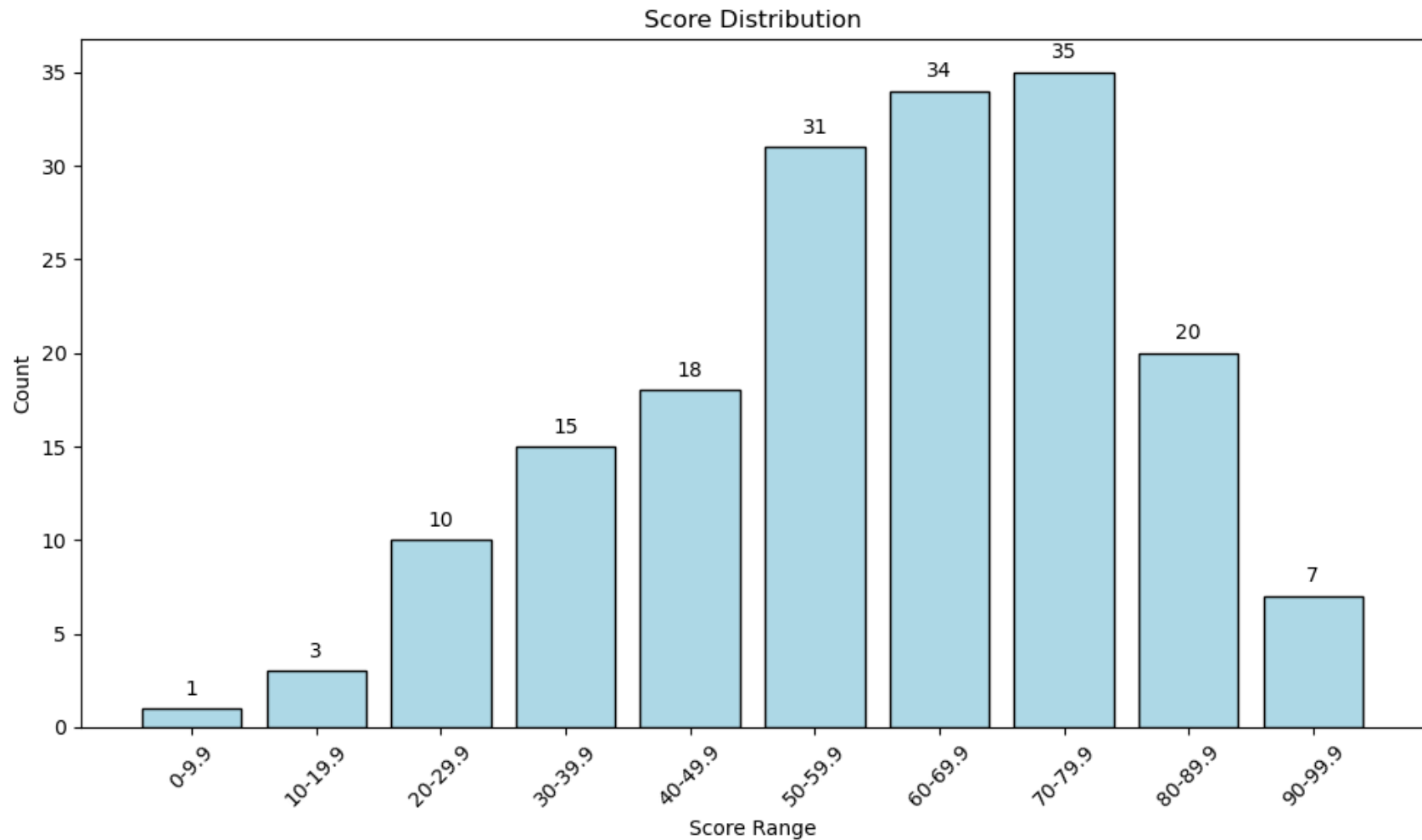
陈俊仕

cjuns@ustc.edu.cn
2025 Fall

计算机科学与技术学院
School of Computer Science and Technology



平均分**60.26** 中位数**64.5**





1 Review

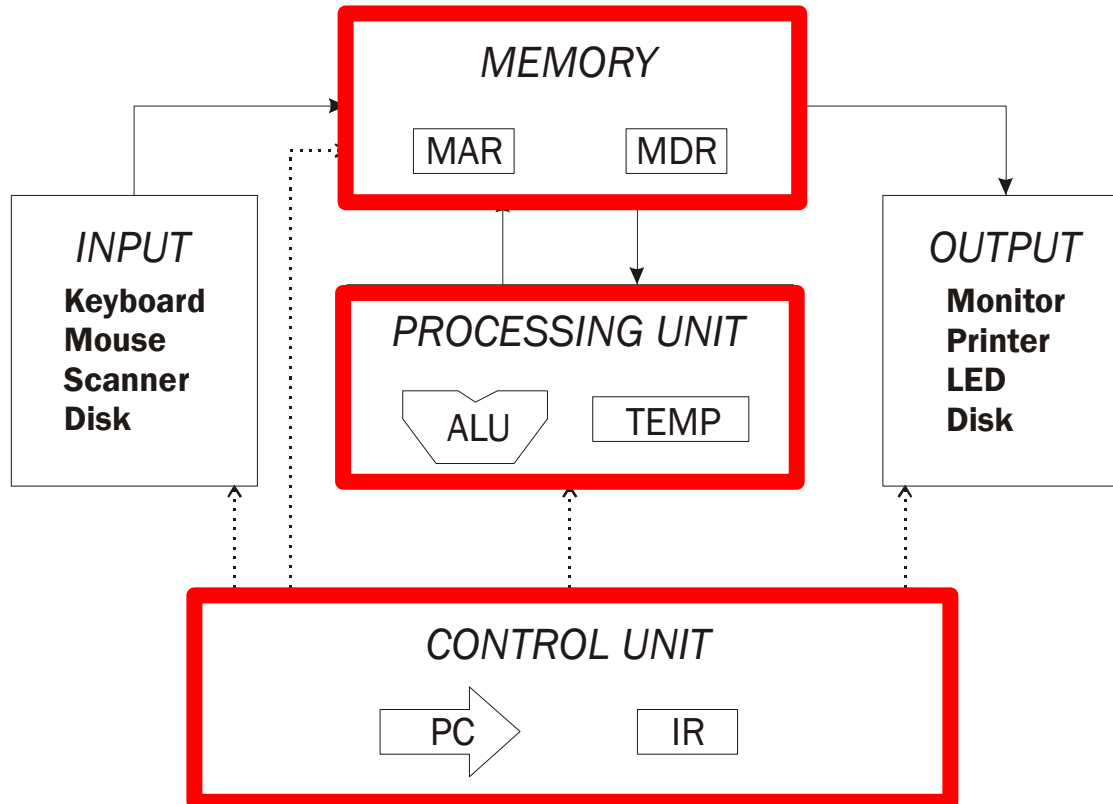
2 The Memory Address Space

3 Input/Output

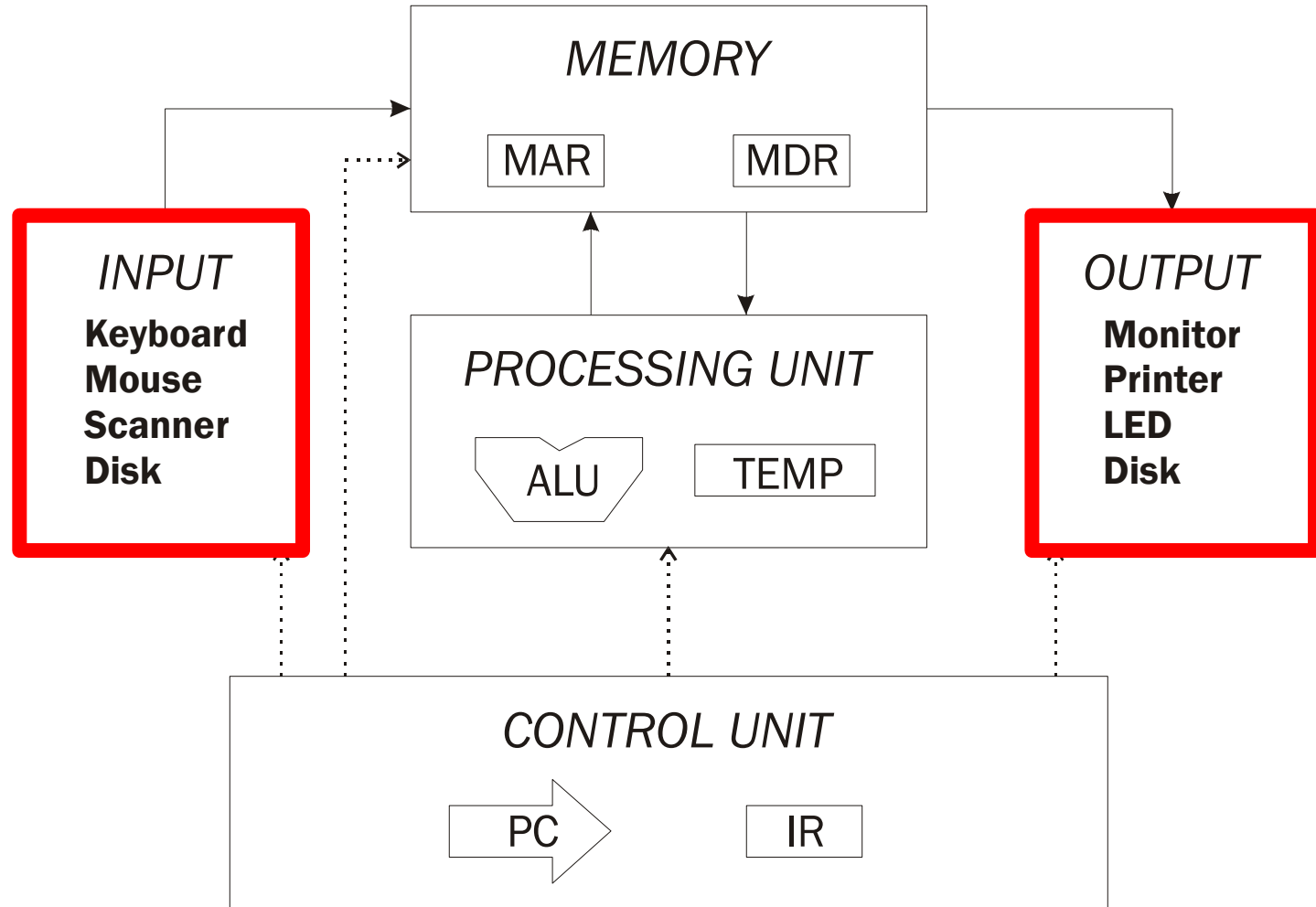
Review

■ So far, we've learned how to:

- compute with values in registers
- load data from memory to registers
- store data from registers to memory



Today: I/O in Von Neumann Model





1 Review

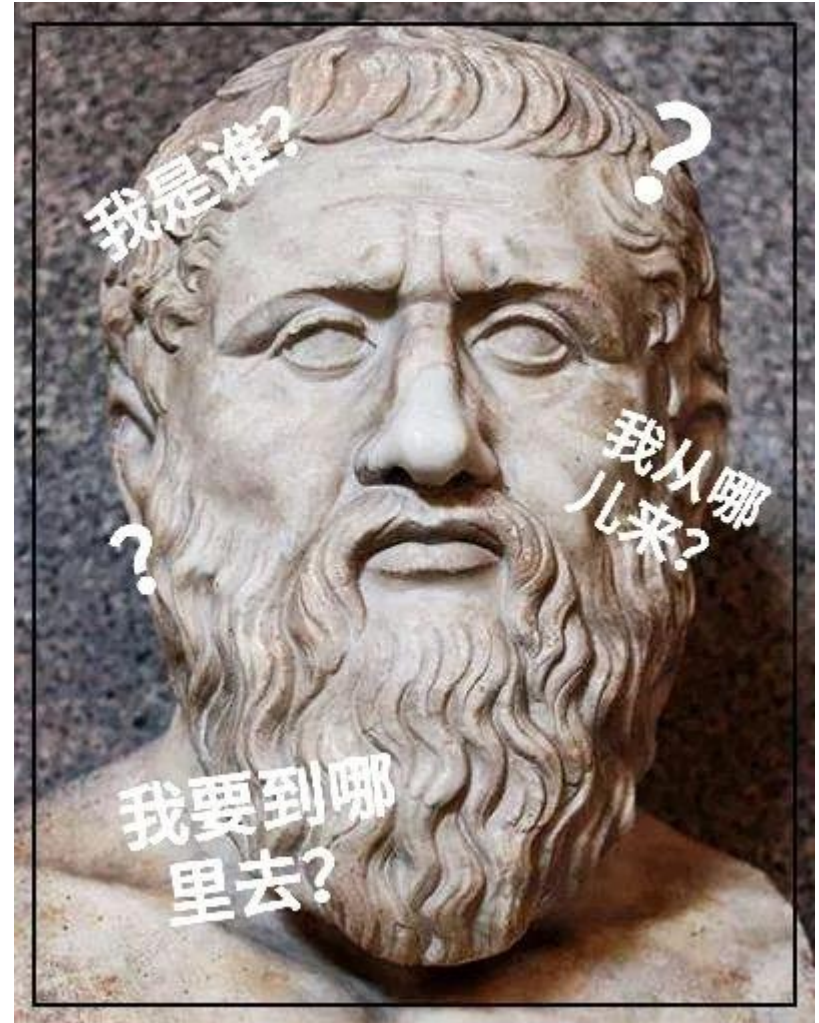
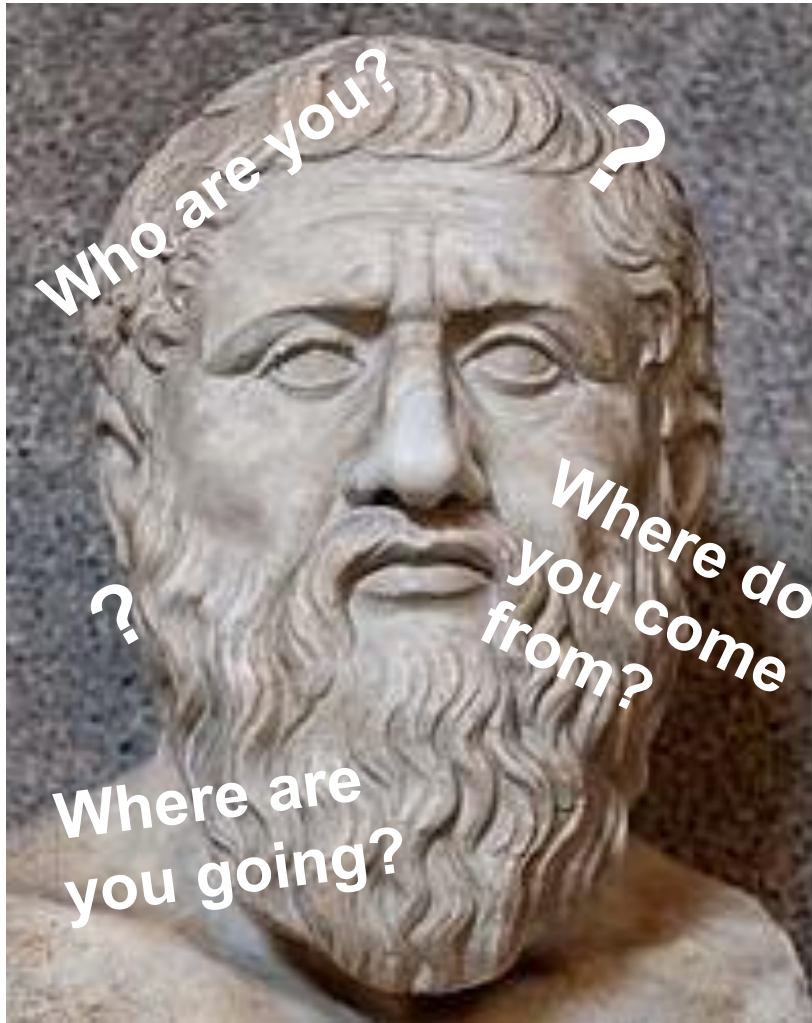
2 The Memory Address Space

3 Input/Output

Philosophy



中国科学技术大学
University of Science and Technology of China



Need for I/O

- But where does data in memory come from? And how does data get out of the system so that humans can use it?
 - Computer systems are useless unless they can process information from outside of the computer and output results outside of the computer
 - I/O is effectively the communication with outside of the computer
- **I/O device** itself communicates with outside world
 - E.g., keyboard takes input from user
- Computer needs to communicate with **I/O device**
 - E.g., computer takes input from keyboard
- **Communication through shared memory locations**
 - Processor and I/O can read/write those memory locations
 - Sometimes, data in memory locations can be set/cleared automatically (by hardware) depending on a read/write

I/O: Connecting to the Outside World

■ Examples

- **Keyboard/mouse input, video output** on a standard computer
- **Network input/output** that enables web surfing
- Information **from an engine of a car** that a computer uses to determine how to tune the engine (output from computer tunes the engine)
- Requests for **airline reservations** and replies that service those requests

■ Types of I/O devices characterized by:

- **behavior**: input, output, storage
 - input: keyboard, motion detector, network interface
 - output: display screen(monitor), printer, network interface
 - storage: disk, CD-ROM
- **data rate**: how fast can data be transferred?
 - keyboard: 100 bytes/sec
 - disk: 60-120 MB/s
 - network: 1 Mb/s - 100 Gb/s

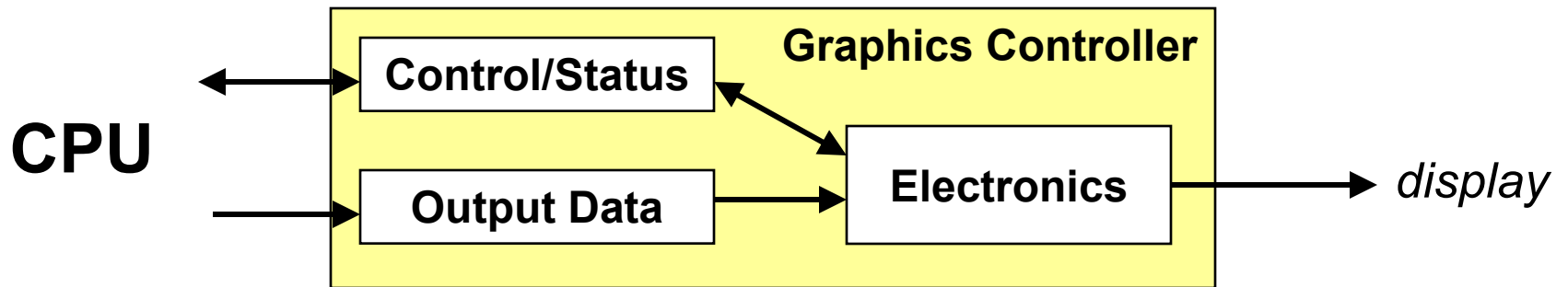
I/O Controller

■ Control/Status Registers

- CPU tells device what to do -- write to control register
- CPU checks whether task is done -- read status register

■ Data Registers

- CPU transfers data to/from device



■ Device electronics

- performs actual operation
 - pixels to screen, bits to/from disk, characters from keyboard

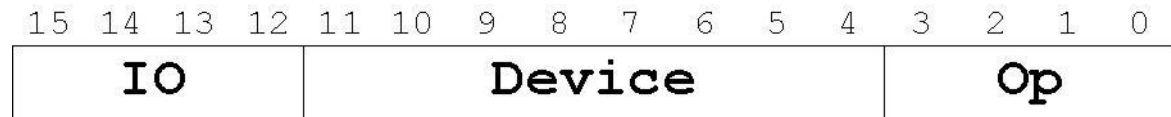
Programming Interface

- How are **device registers** identified?
 - Memory-mapped vs. special instructions
- How is timing of transfer managed?
 - Asynchronous vs. synchronous
- Who controls transfer?
 - CPU (**polling**) vs. device (**interrupts**)

Memory-Mapped vs. I/O Instructions

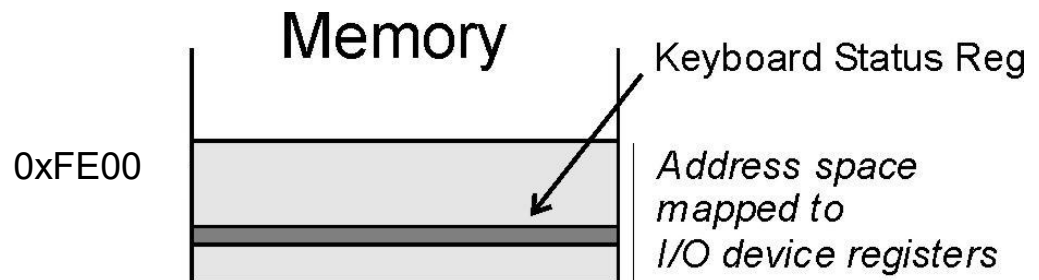
■ Instructions

- PDP-8 (from DEC, 1965)
- designate opcode(s) for I/O
- register and operation encoded in instruction



■ Memory-mapped

- assign a memory address to each device register
- use data movement instructions (LD/ST) for control and data transfer



Transfer Timing

■ I/O events generally happen much slower than CPU cycles.

- If : CPU 300MHz, 10clocks/character, 6characters/word
- Then: typing speed $(300 \times 10^6) / (10 \times 6) = 5 \times 10^6$ words/s

■ Synchronous

- data supplied at a fixed, predictable rate
- CPU reads/writes every X cycles

■ Asynchronous

- data rate less predictable
- CPU must synchronize with device,
so that it doesn't miss data or write too quickly

Transfer Control

- Who determines when the next data transfer occurs?
 - CPU vs. I/O device
- **Polling** is explicitly looking/examining
 - CPU keeps checking status register until new data arrives OR device ready for next data
 - “Are you there yet? Are you there yet? Are you there yet?”
- **Interrupts** is a nudge, knock on the door, loud noise, which forces you to pay attention
 - Device sends a special signal to CPU when new data arrives OR device ready for next data
 - CPU can be performing other tasks instead of polling device.
 - “Wake me when you get there.”

■ Memory-mapped I/O (Table A.1)

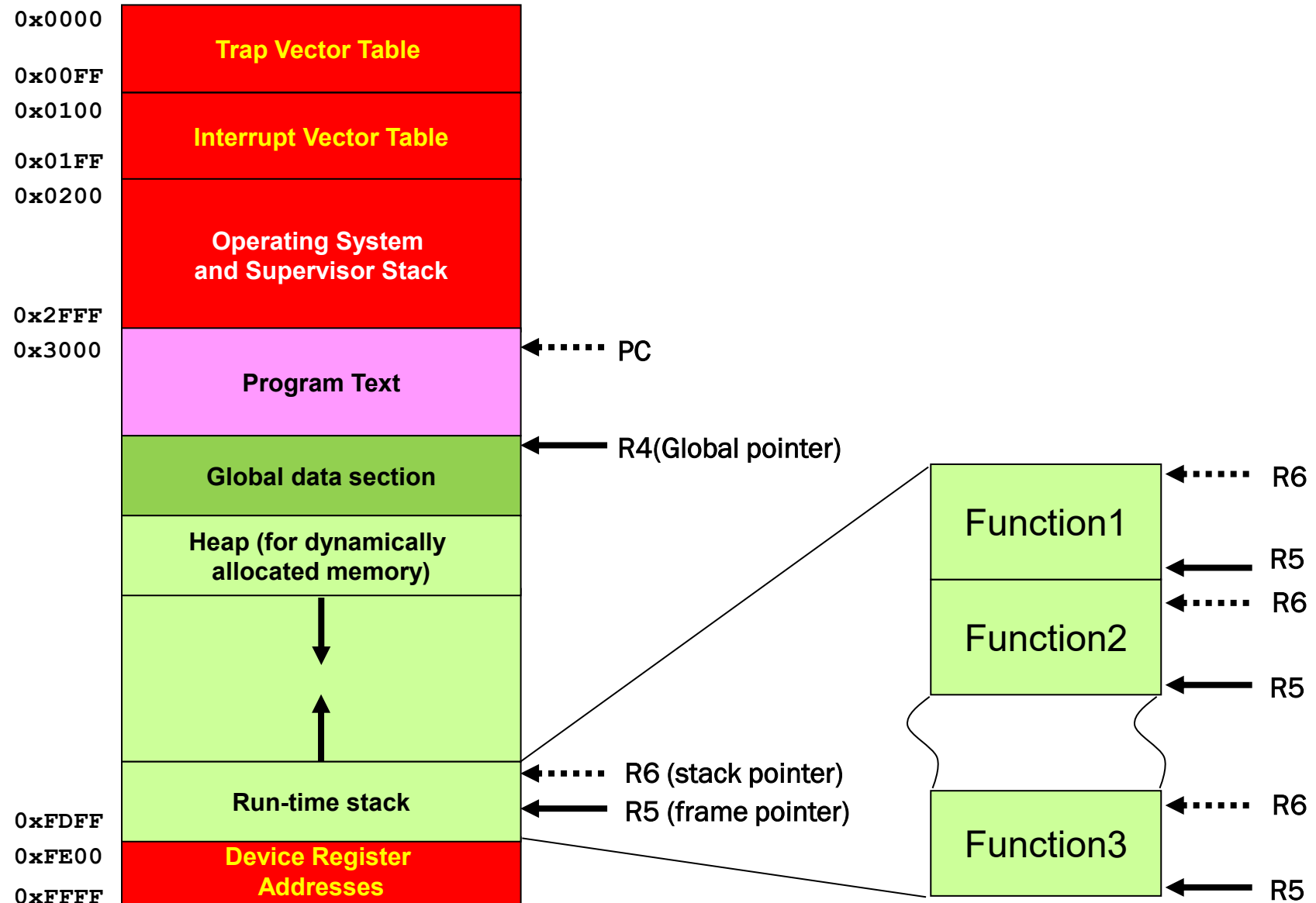
<i>Location</i>	<i>I/O Register</i>	<i>Function</i>
xFE00	Keyboard Status Reg (KBSR)	Bit [15] is one when keyboard has received a new character.
xFE02	Keyboard Data Reg (KBDR)	Bits [7:0] contain the last character typed on keyboard.
xFE04	Display Status Register (DSR)	Bit [15] is one when device ready to display another char on screen.
xFE06	Display Data Register (DDR)	Character written to bits [7:0] will be displayed on screen.

■ Asynchronous devices

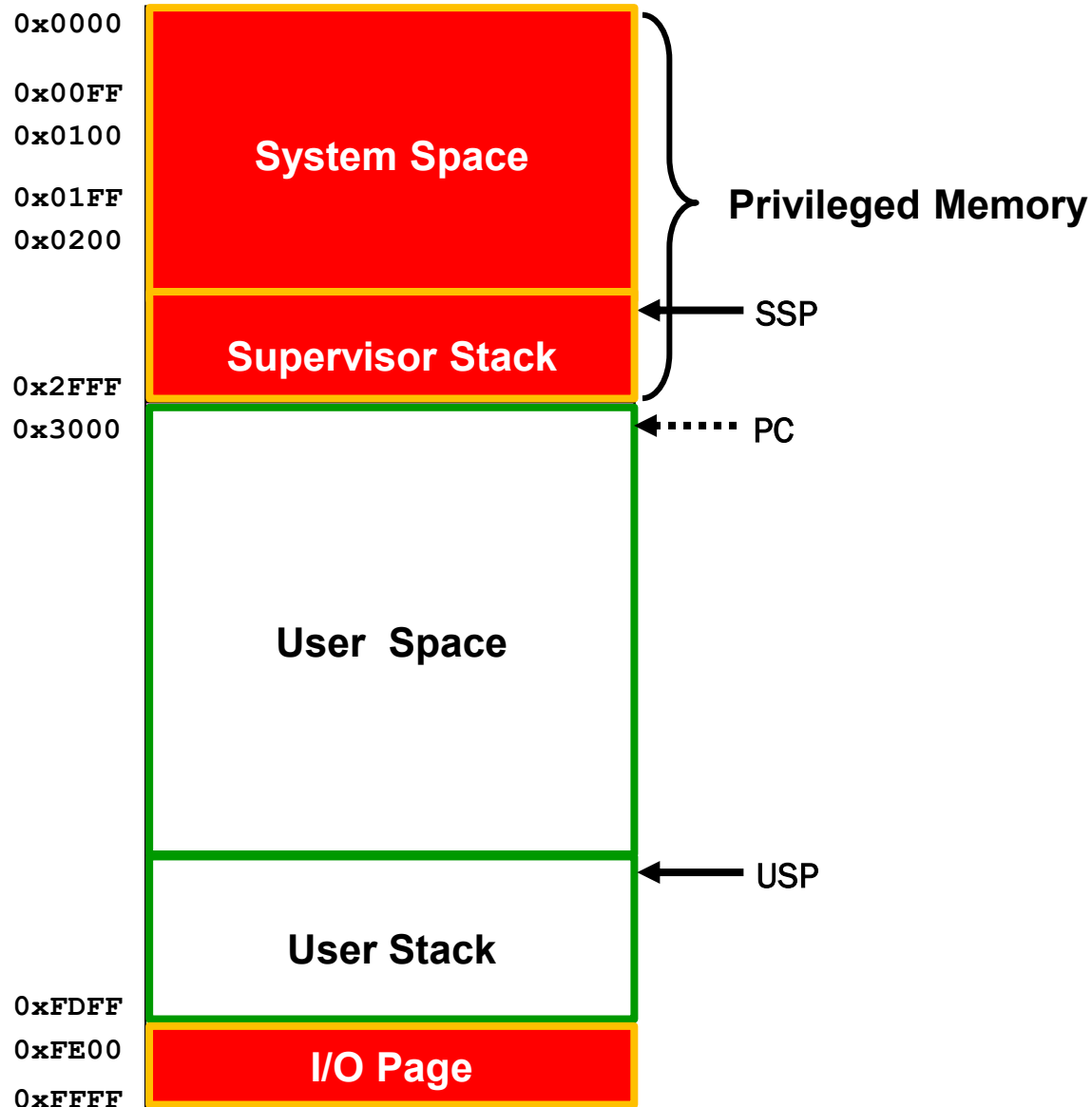
- synchronized through status registers

■ Polling and Interrupts

Memory Map of the LC-3



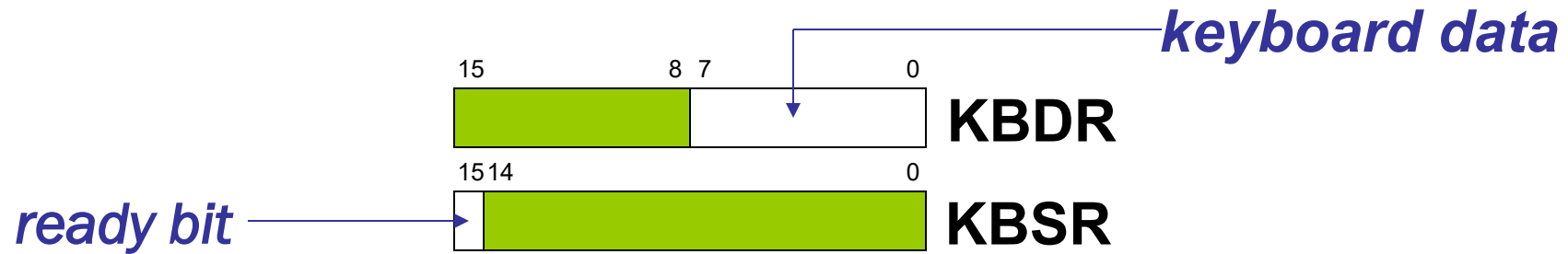
Organization of Memory of the LC-3



Example: Processor **Input from Keyboard**

■ When a character is typed:

- its ASCII code is placed in bits [7:0] of KBDR (bits [15:8] are always zero)
- the “ready bit” (KBSR[15]) is set to one
- keyboard is disabled -- any typed characters will be ignored



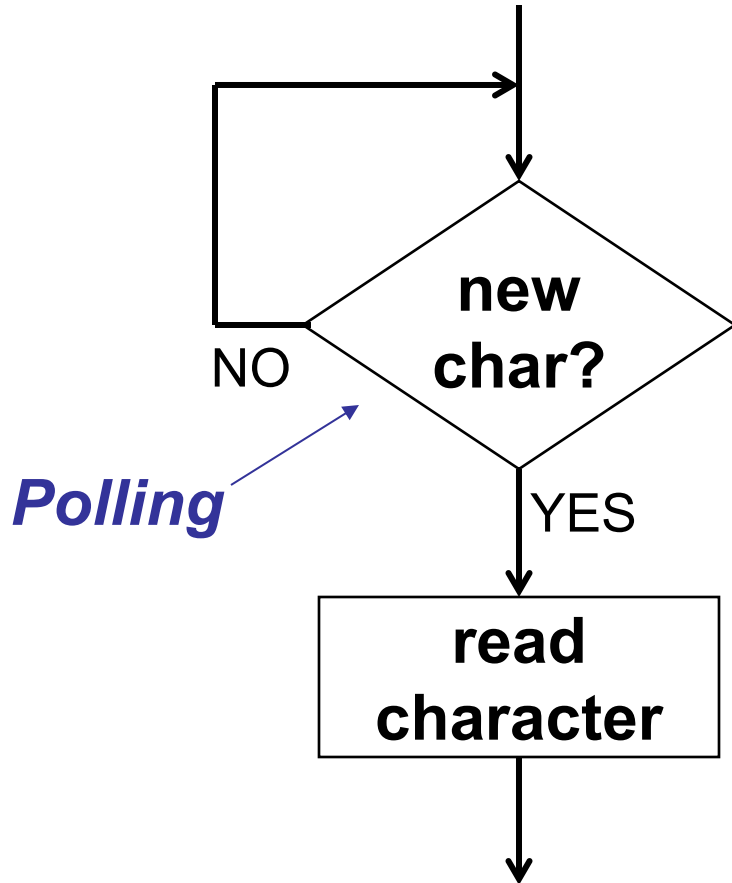
■ When KBDR is read:

- KBSR[15] is set to zero, meaning no keyboard key is pending
- keyboard is enabled

Memory-mapped Operations

- How do we read ready bit?
- How do we test whether the bit is one?
- How do we read keyboard data?

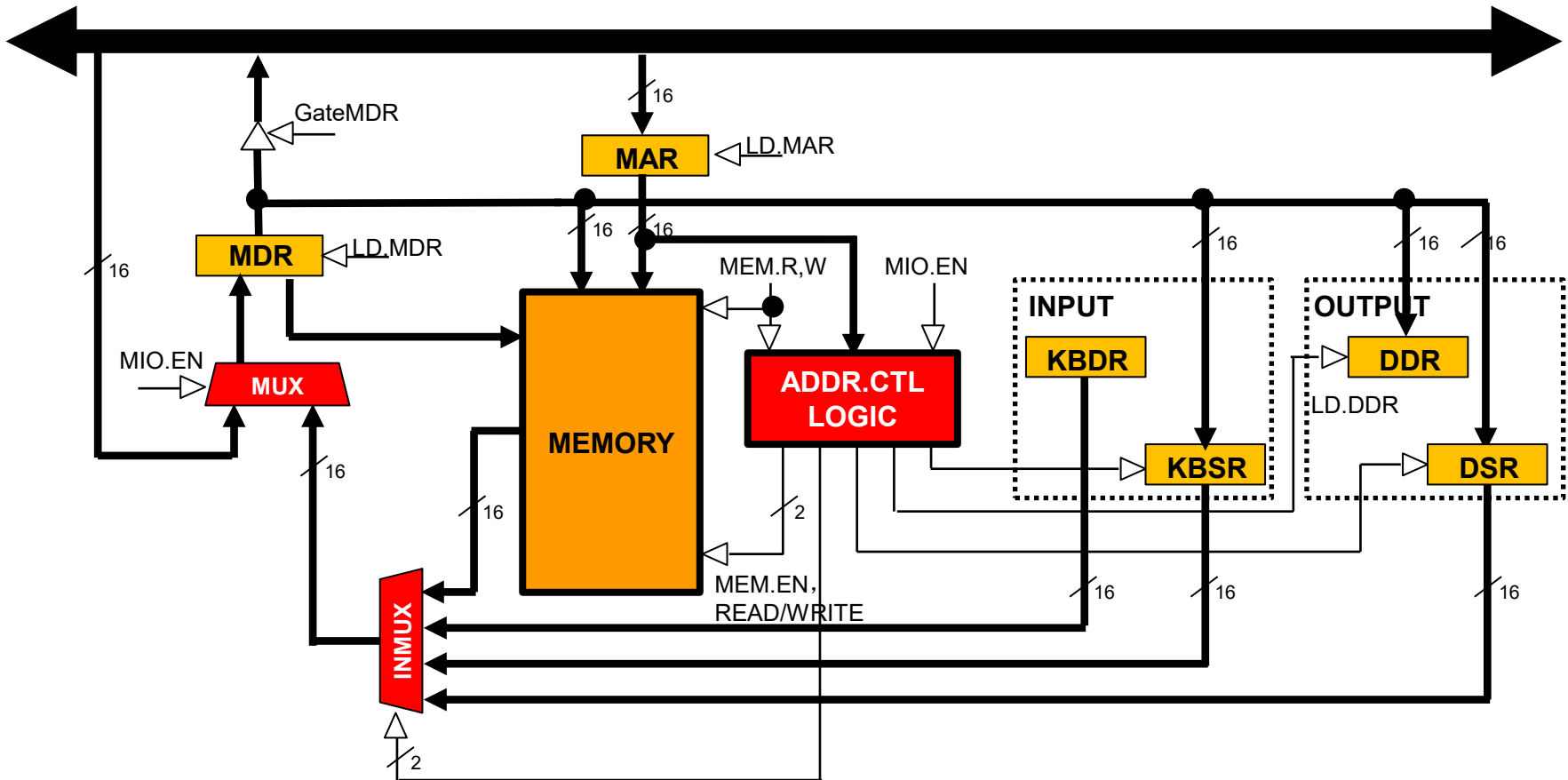
Basic Input Routine



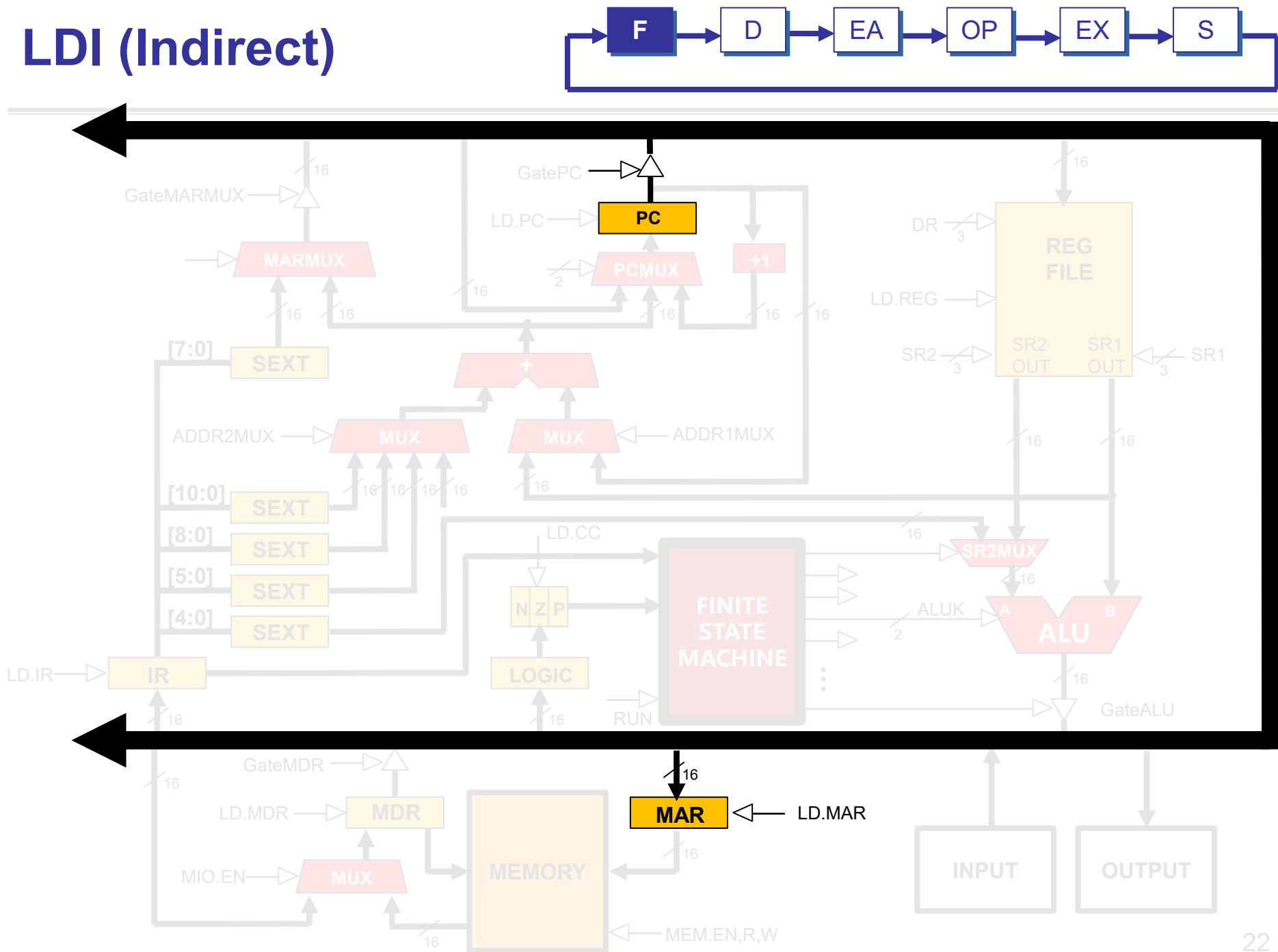
```
POLL    LDI    R0, KBSR
        BRzp   POLL
        LDI    R0, KBDR
        . . .

KBSR    .FILL   xFE00
KBDR    .FILL   xFE02
```

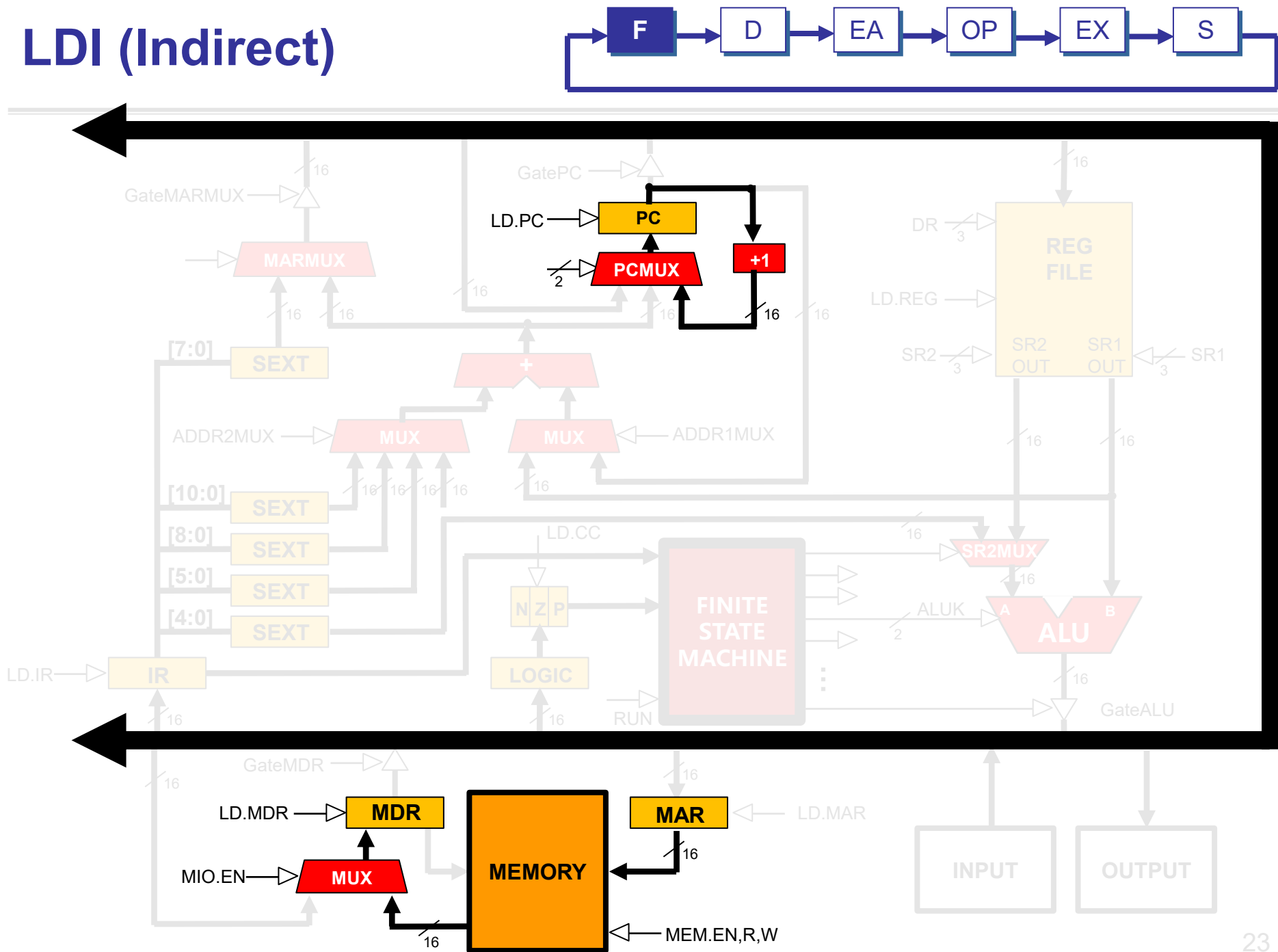
Memory-mapped I/O



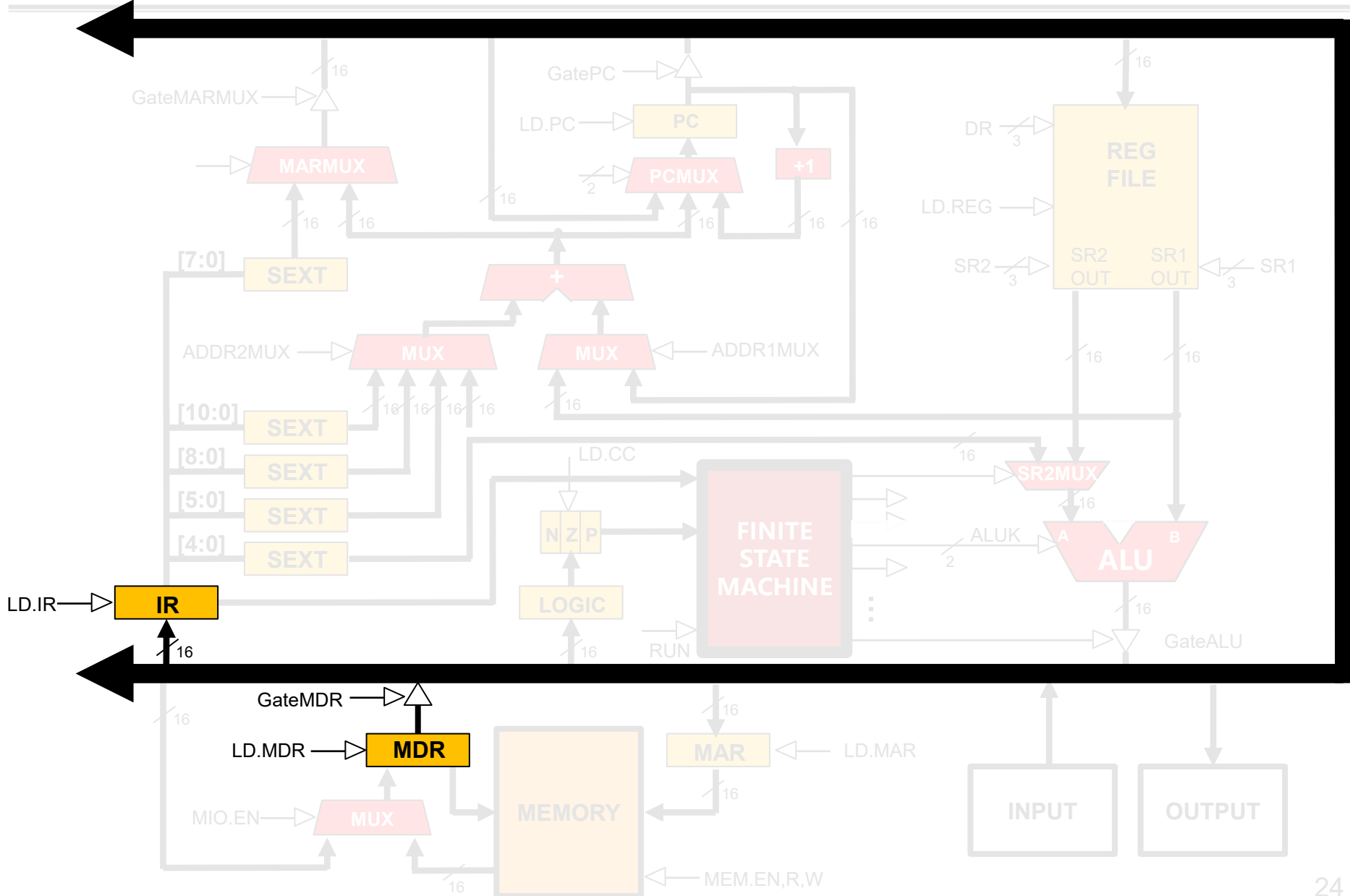
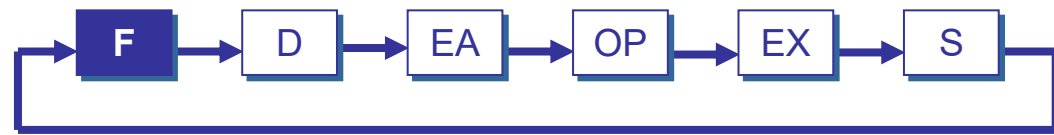
LDI (Indirect)



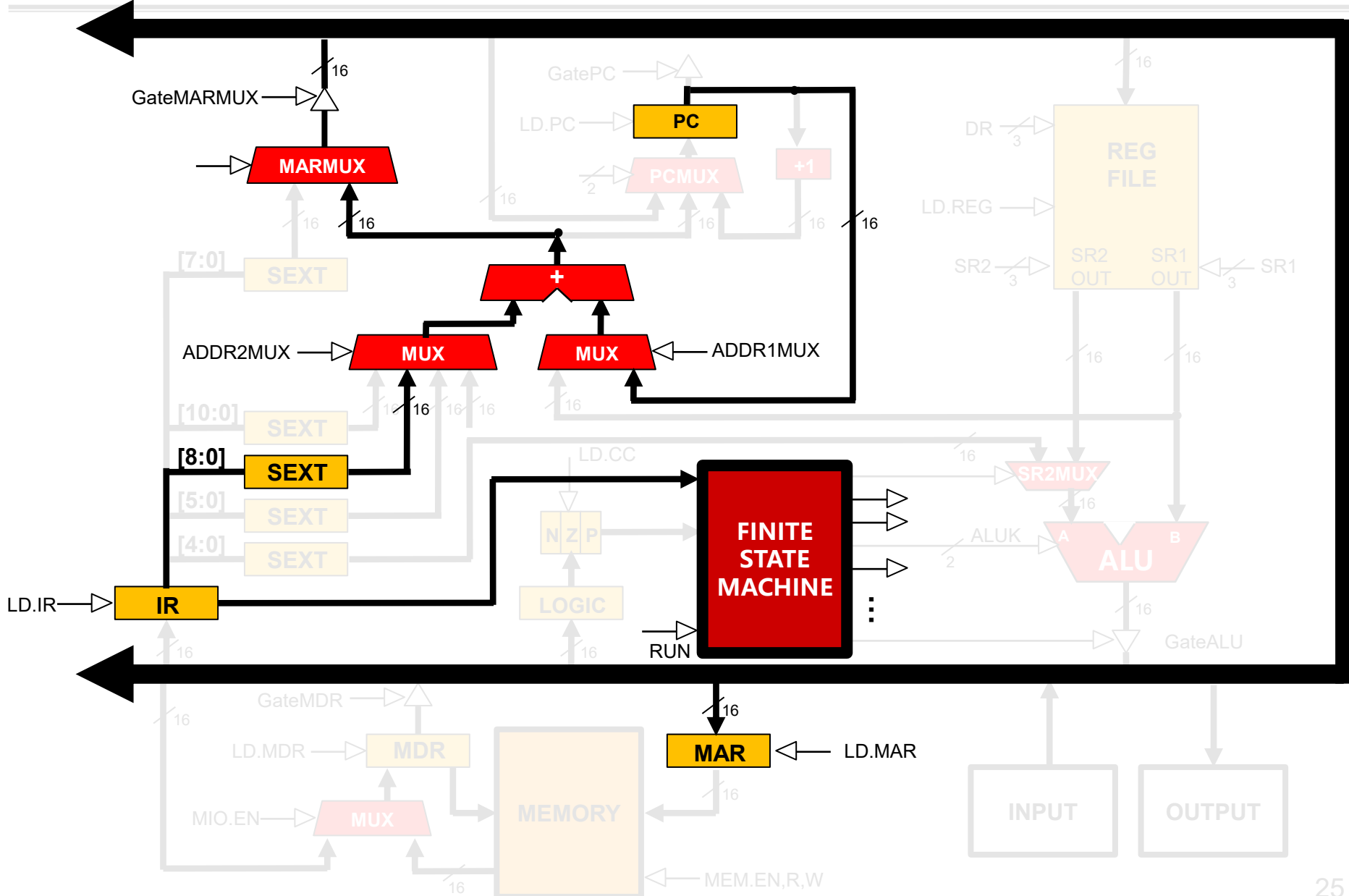
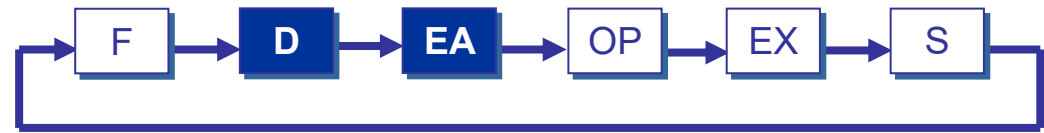
LDI (Indirect)



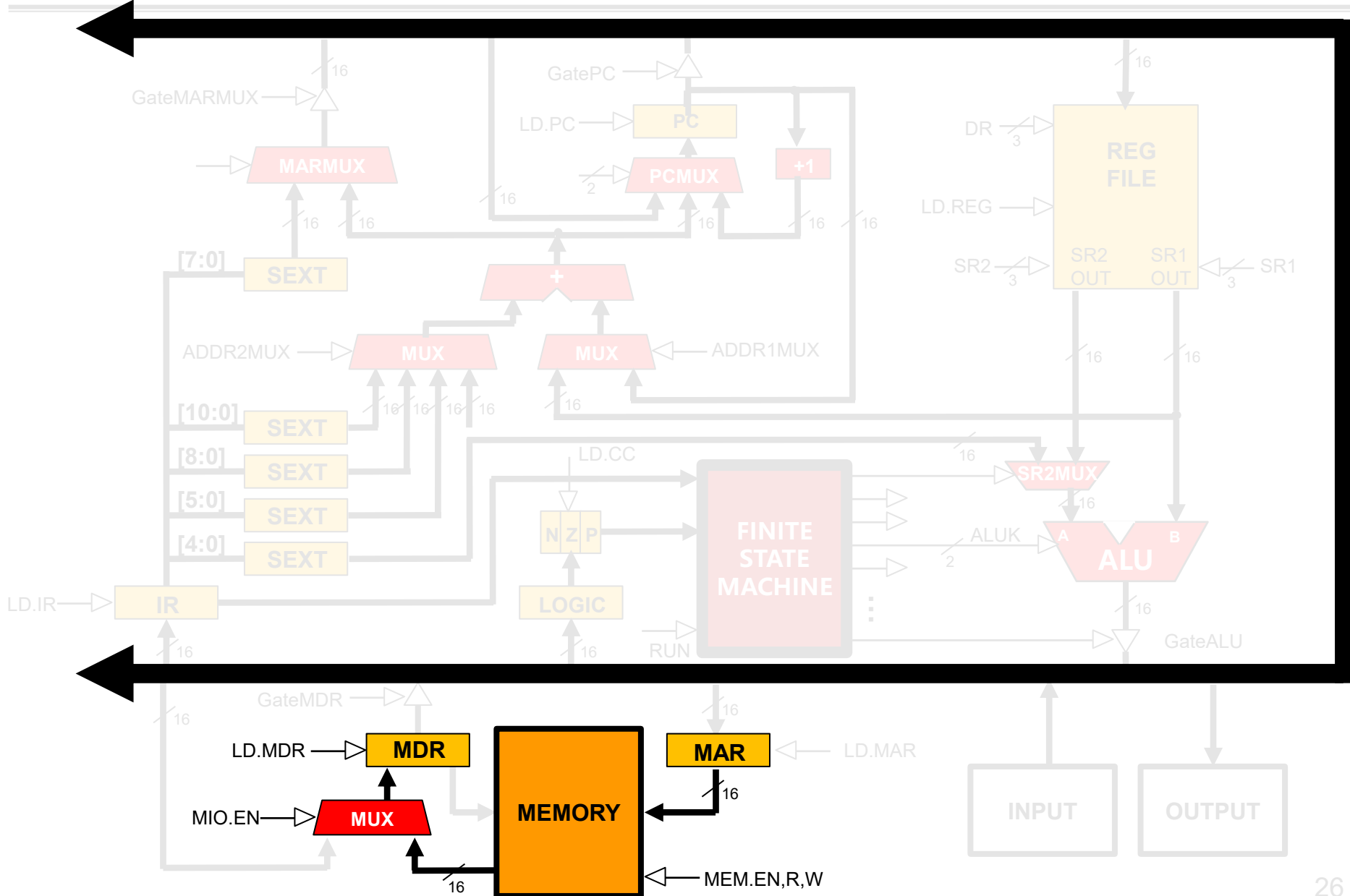
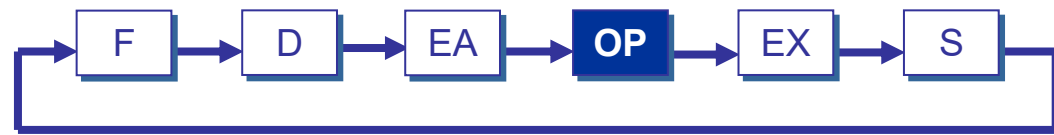
LDI (Indirect)



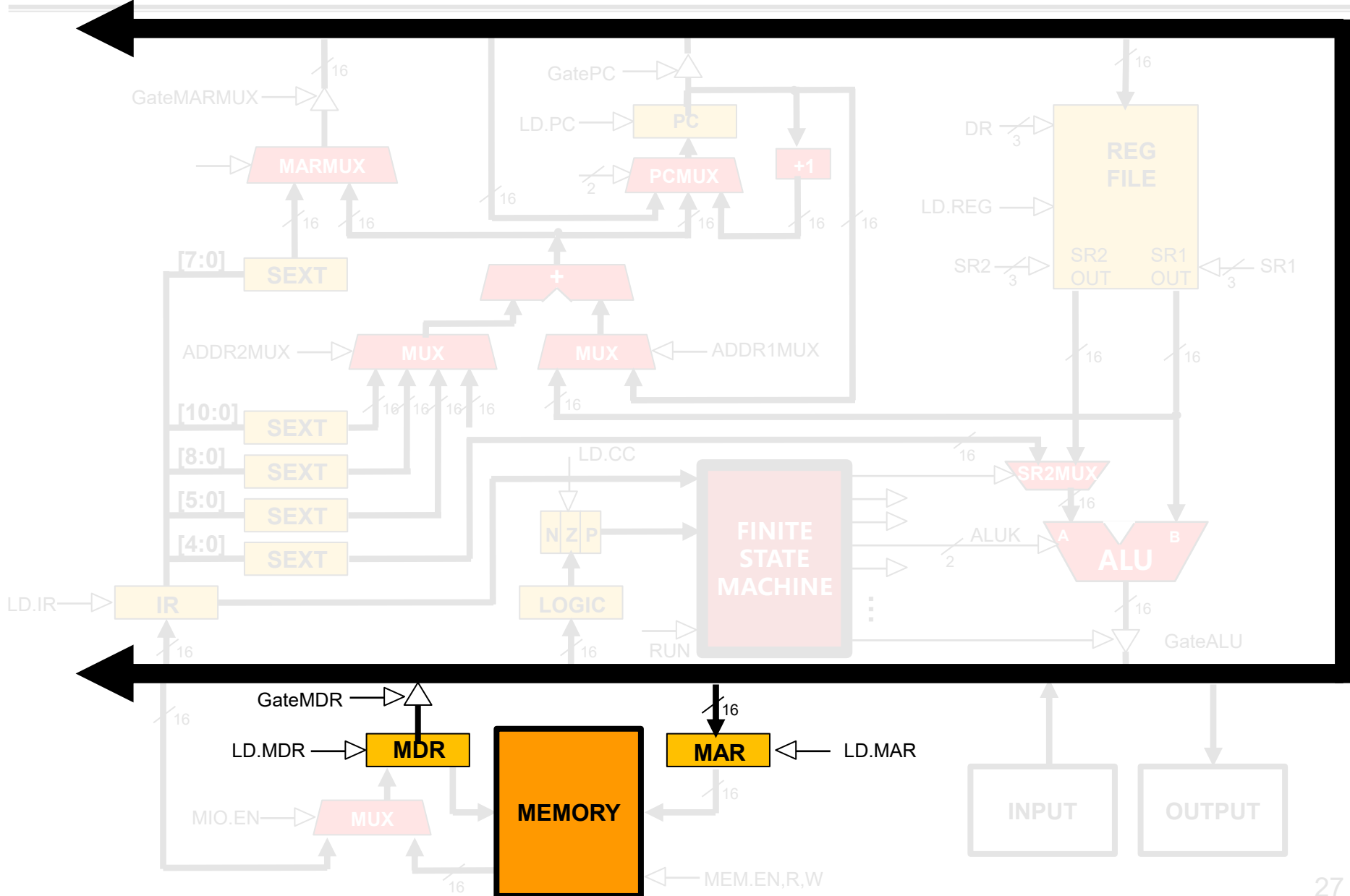
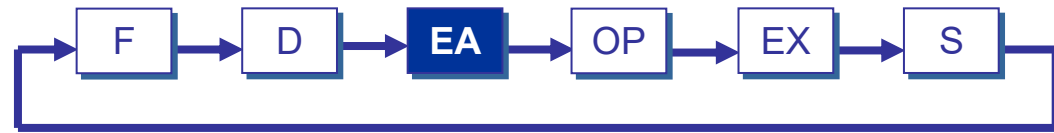
LDI (Indirect)



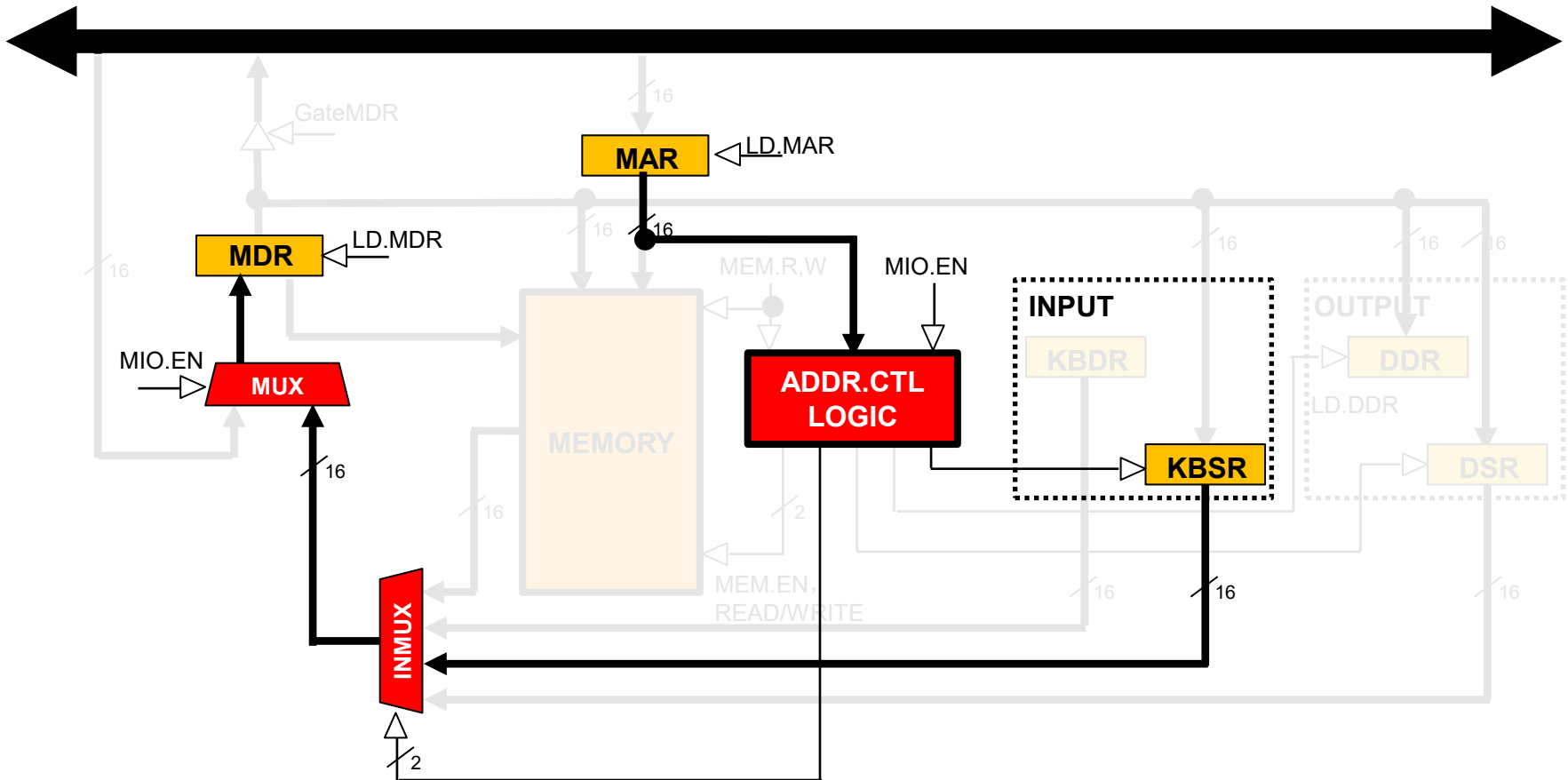
LDI (Indirect)



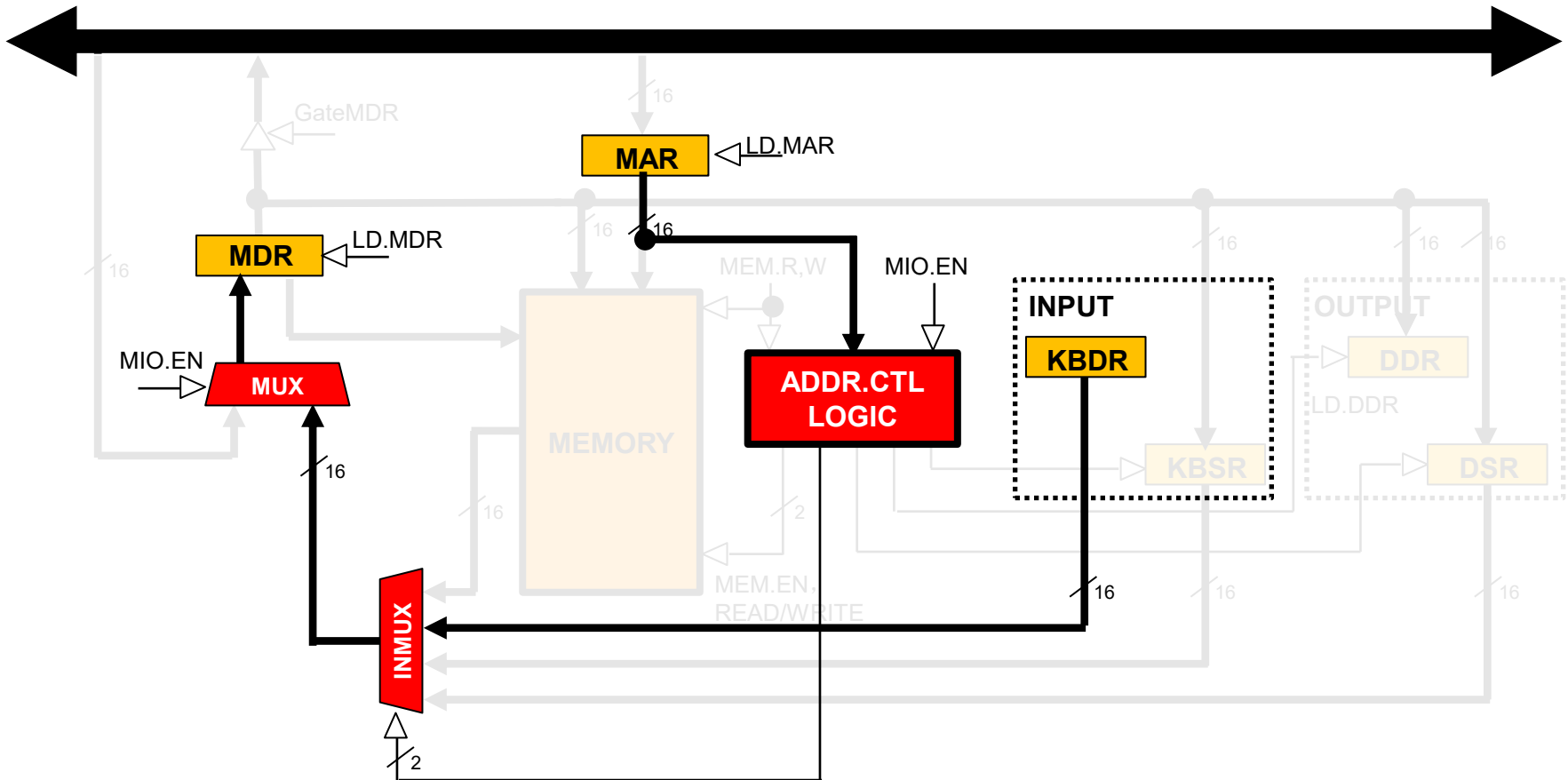
LDI (Indirect)



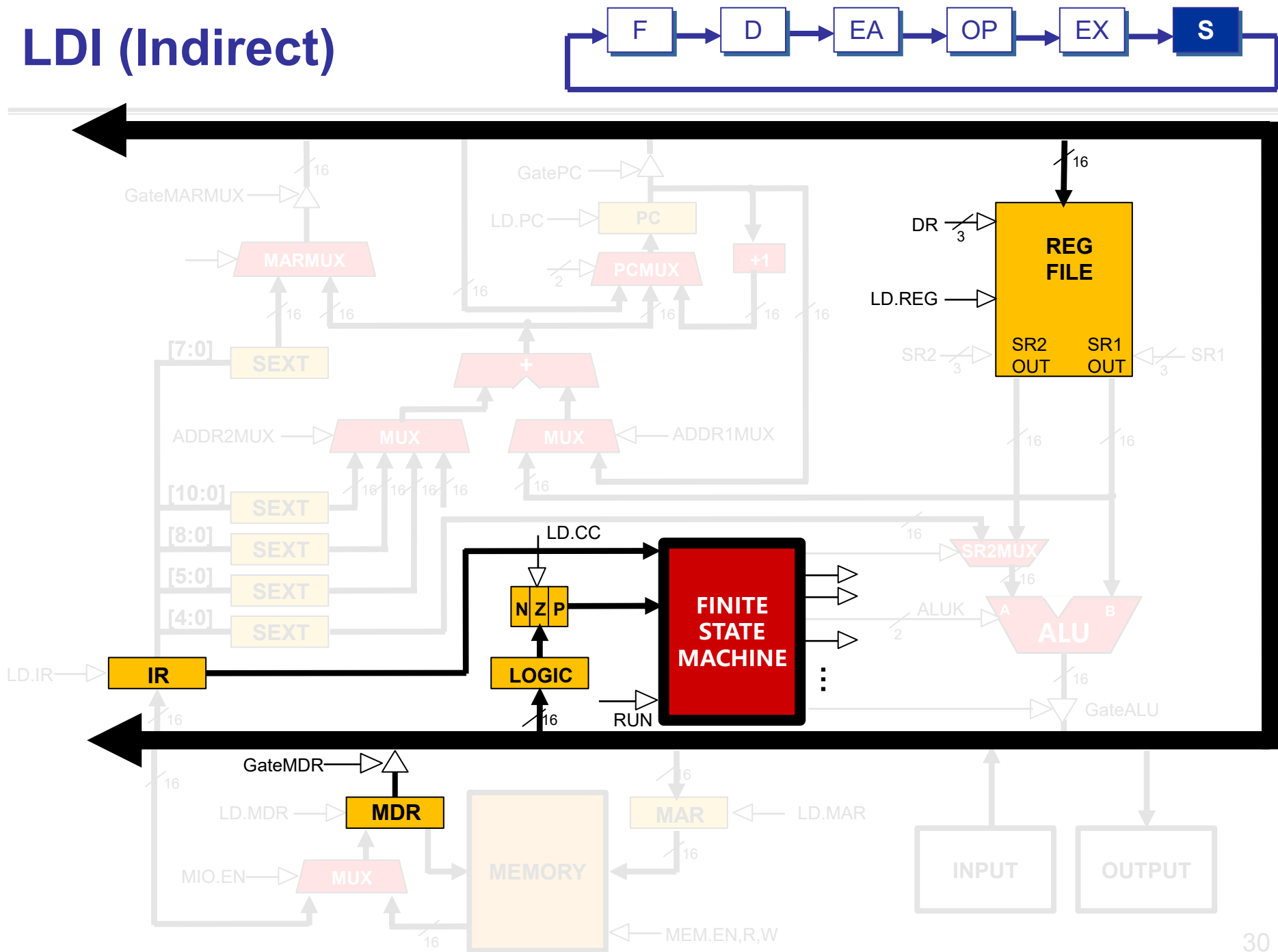
Memory-mapped I/O: `LDI R0, KBSR`



Memory-mapped I/O: `LDI R0, KBDR`



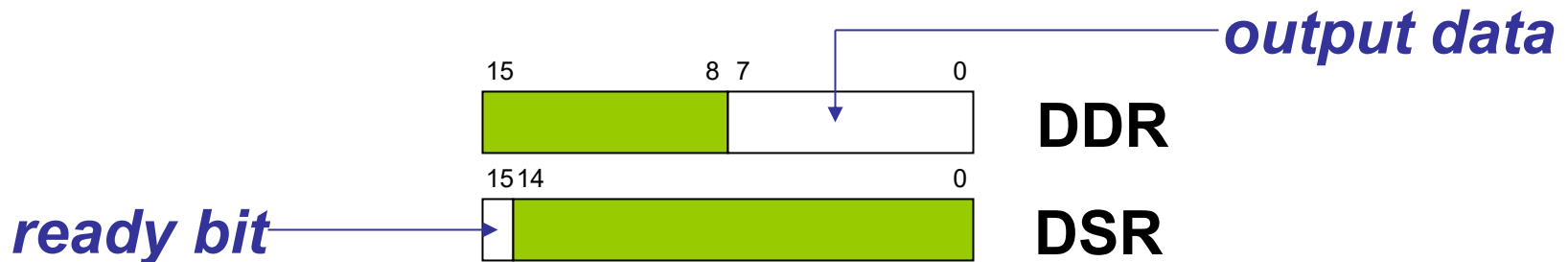
LDI (Indirect)



Example: Processor Output to Screen

■ When Display device is ready to display another character:

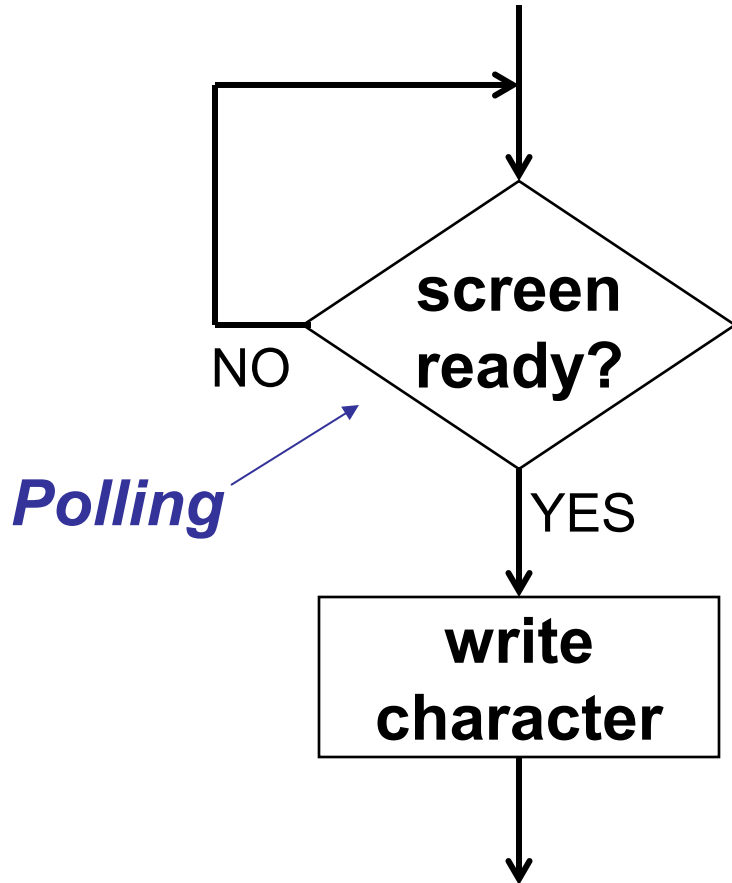
- the “ready bit” (DSR[15]) is set to one, indicating that processor can write a character for display



■ When data is written to the Display data register:

- DSR[15] automatically set to 0
- character in DDR[7:0] is displayed
- any other character data written to DDR is ignored (while DSR[15] is zero)

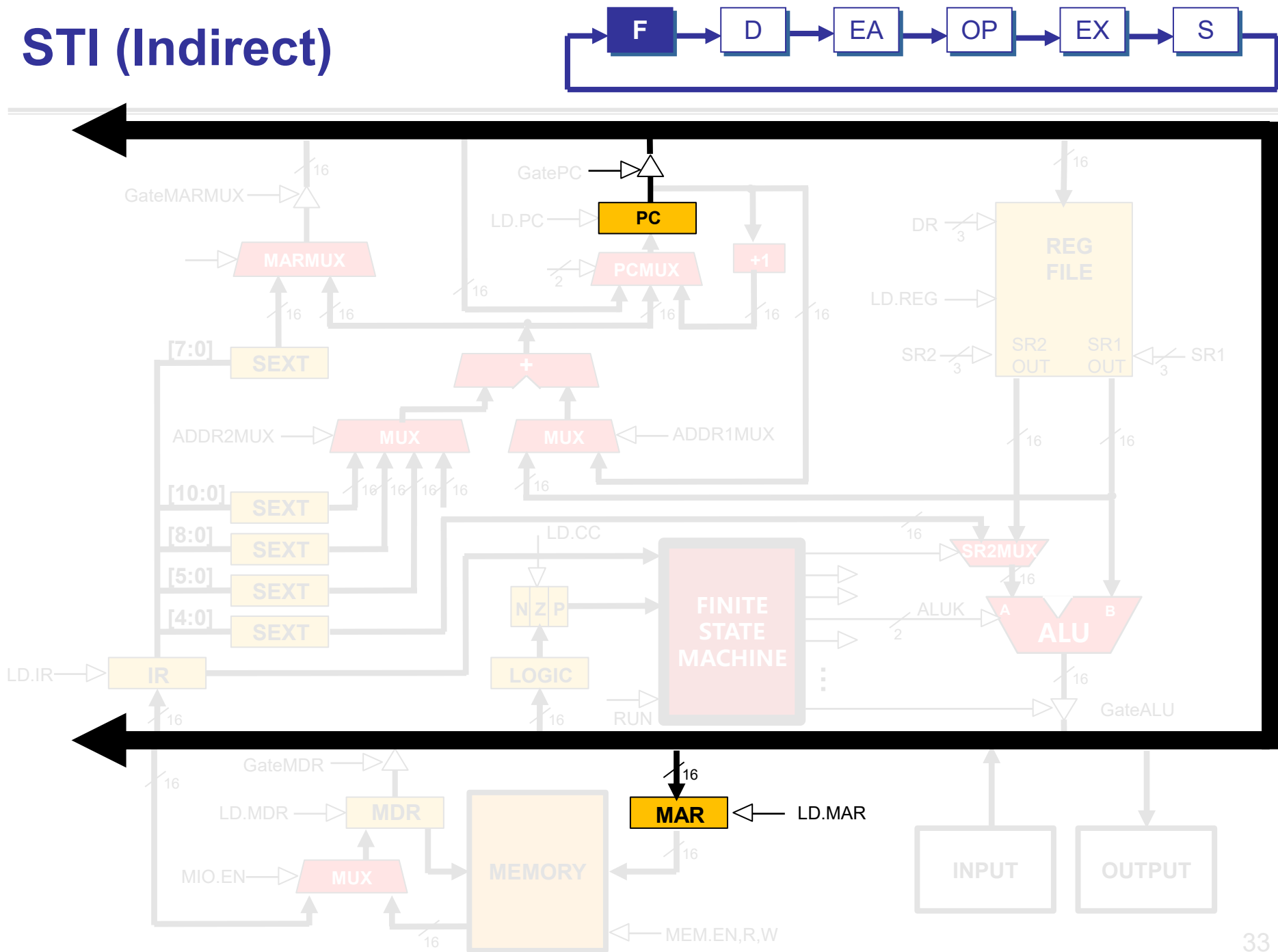
Basic Output Routine



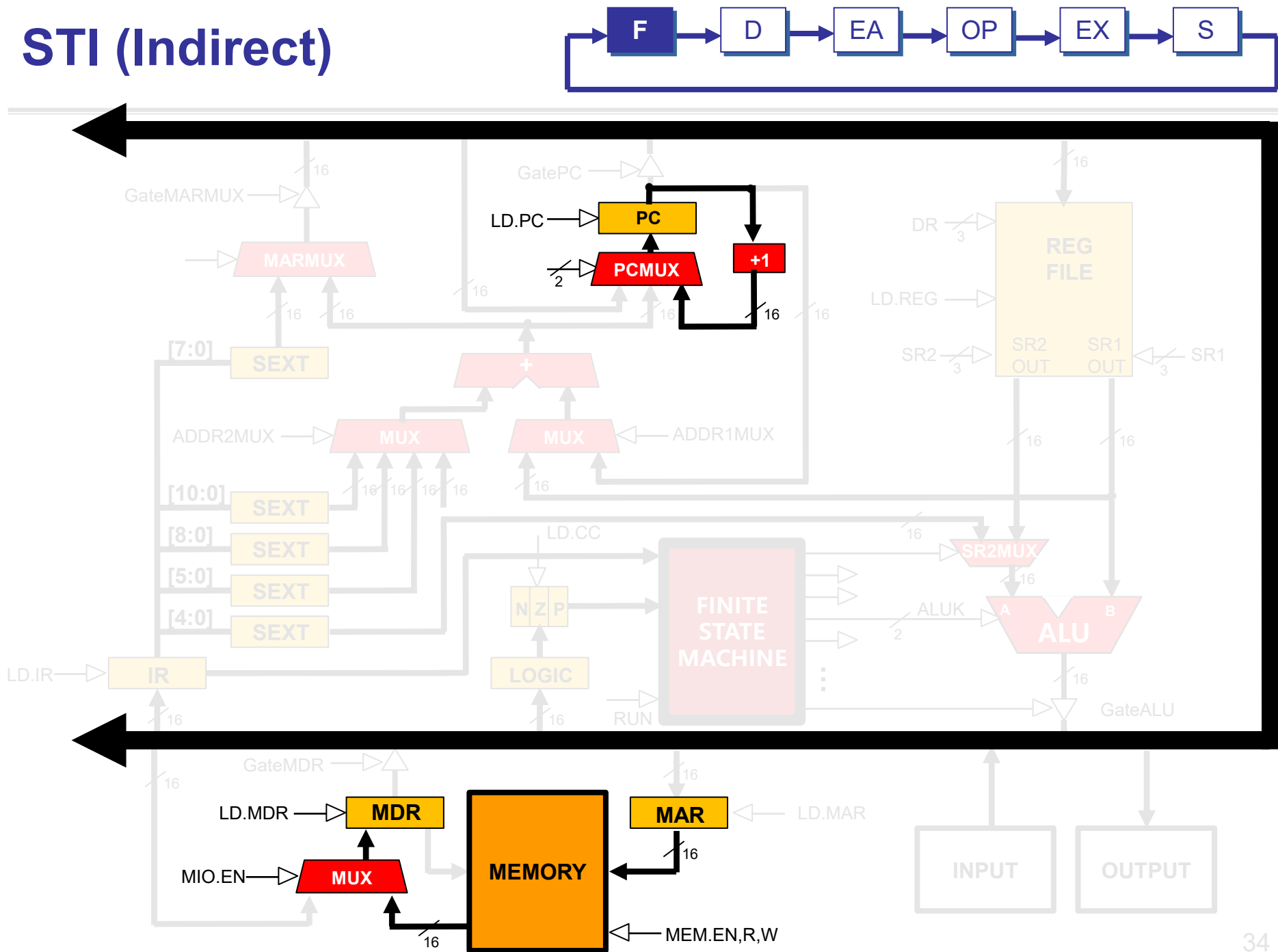
```
POLL    LDI    R1, DSR
        BRzp   POLL
        STI    R0, DDR
        . . .

DSR      .FILL  xFE04
DDR      .FILL  xFE06
```

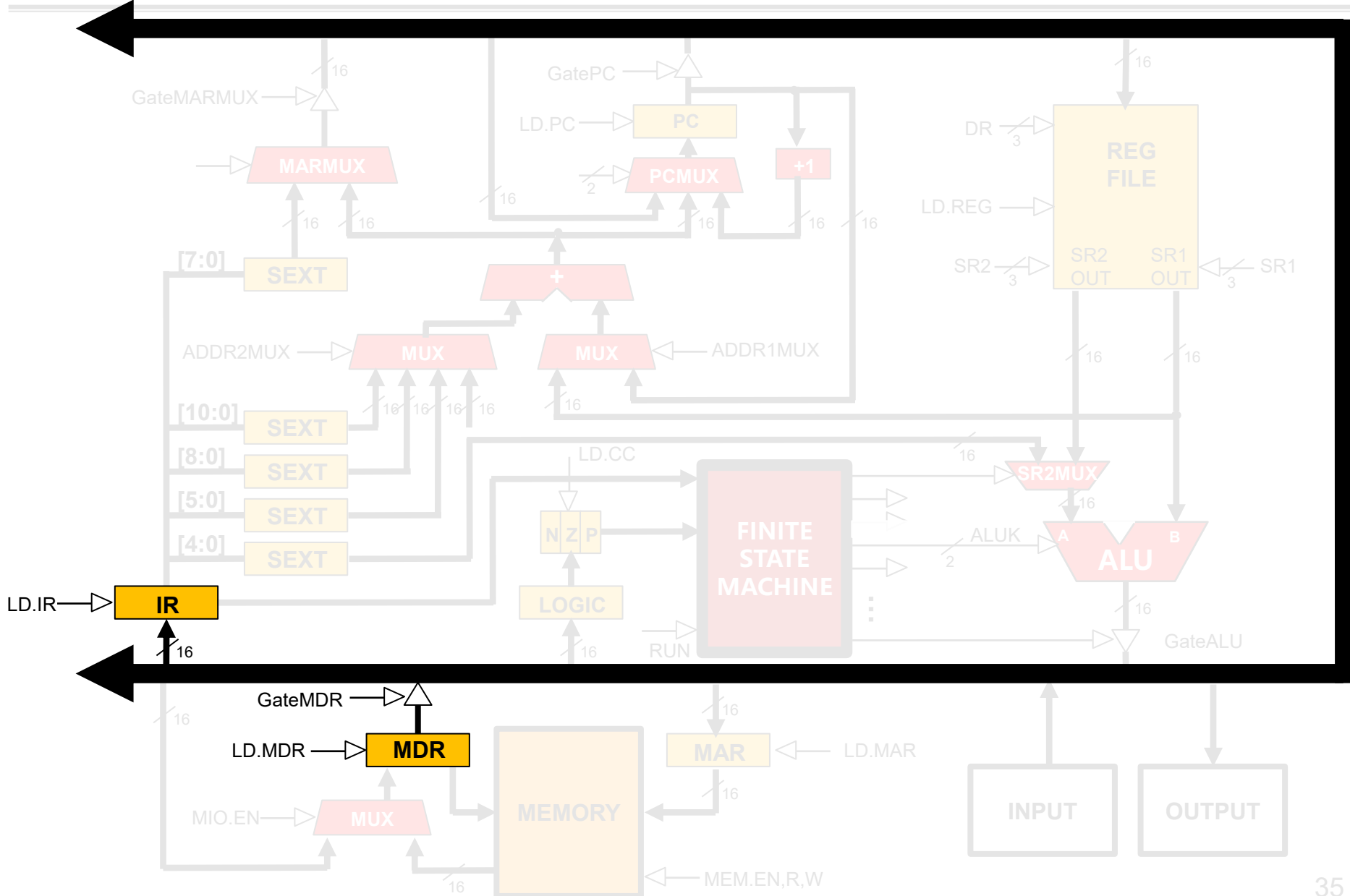
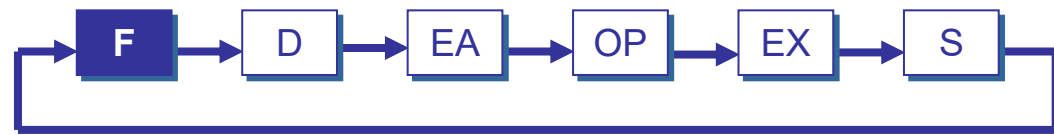
STI (Indirect)



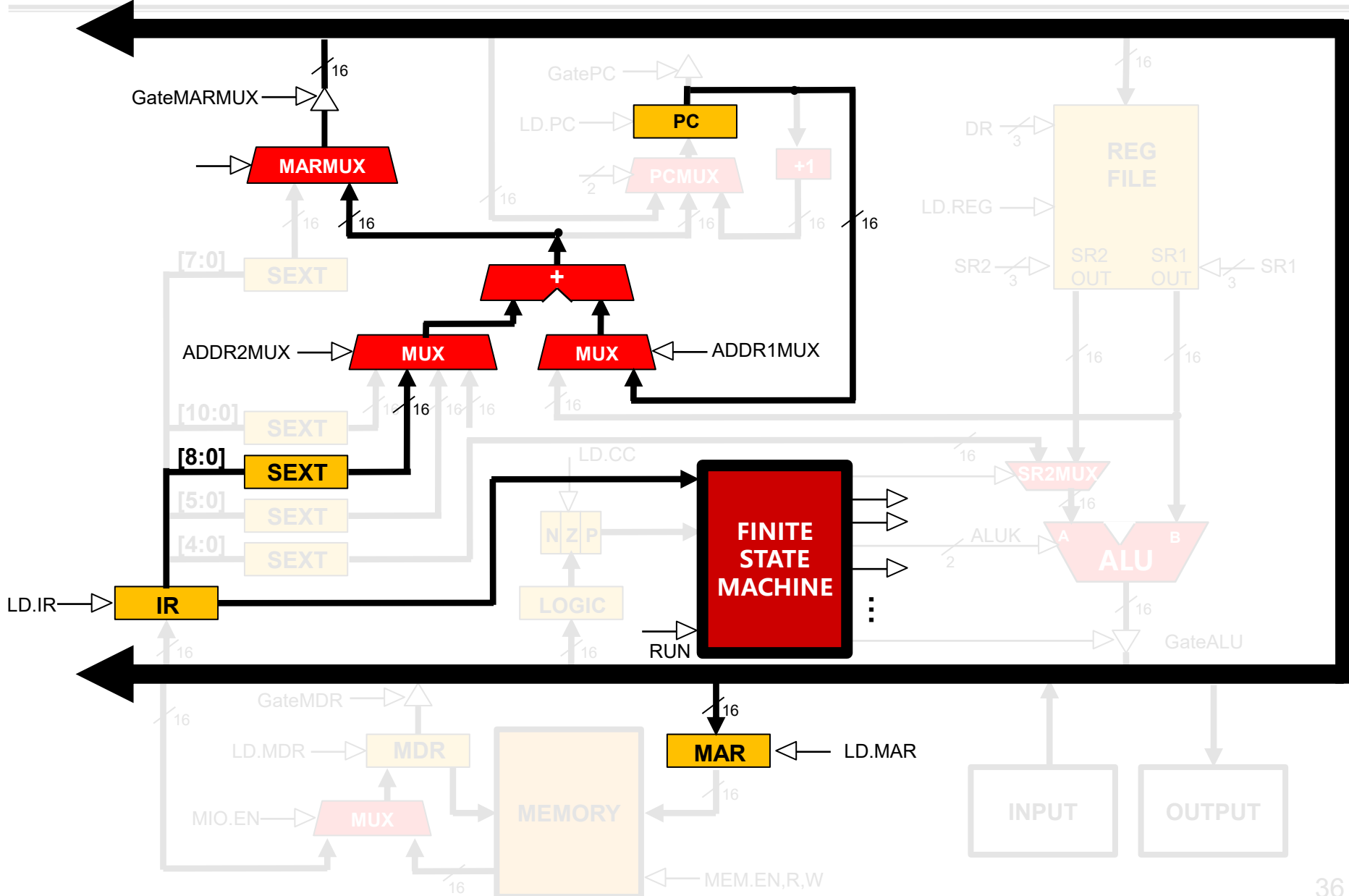
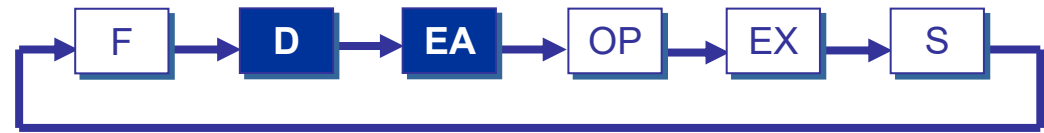
STI (Indirect)



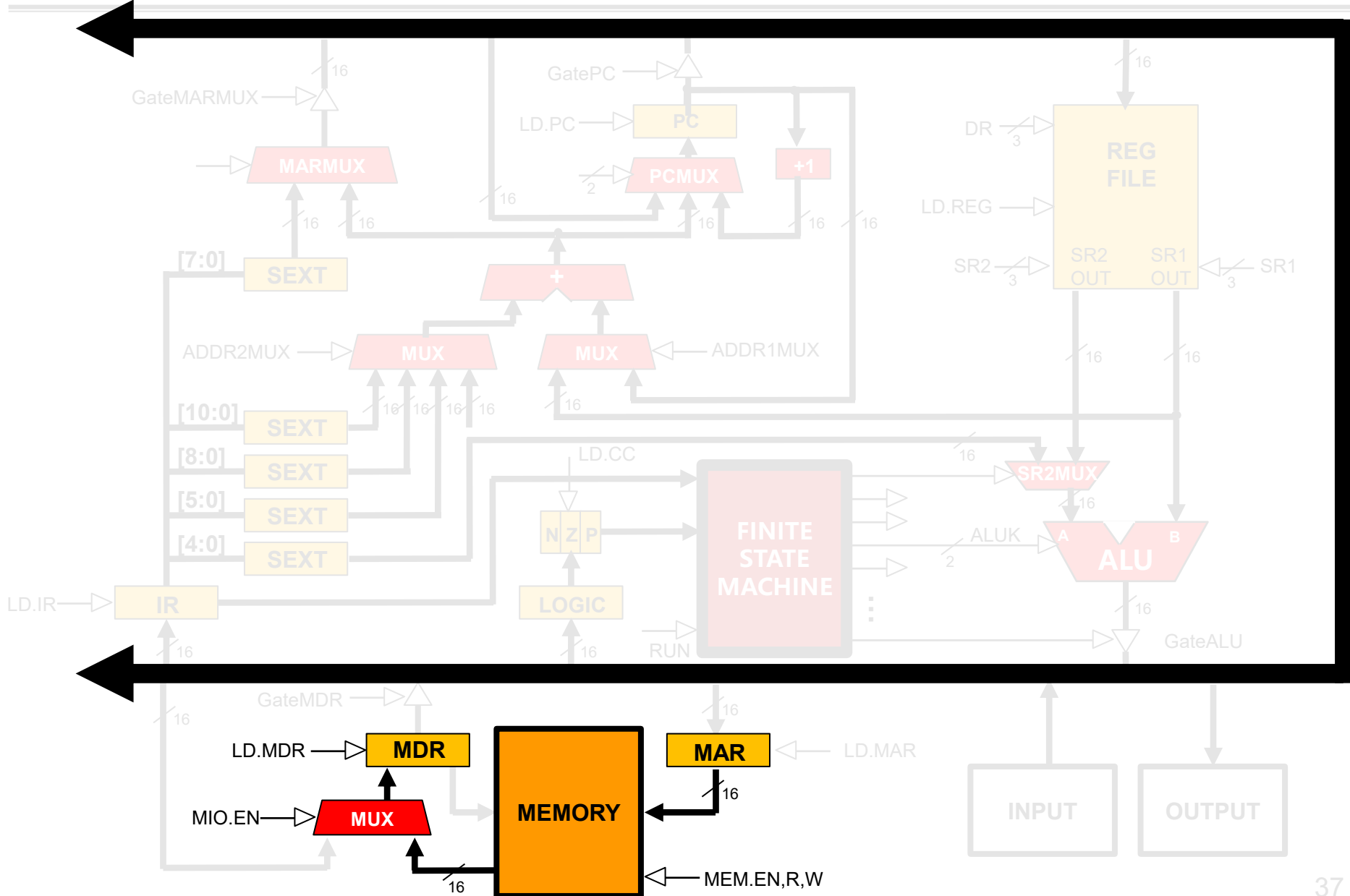
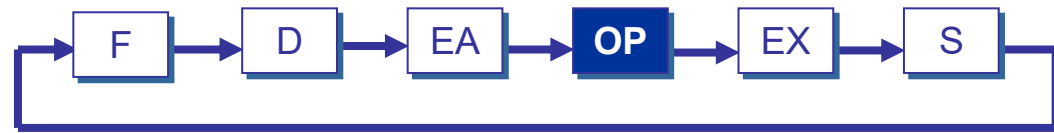
STI (Indirect)



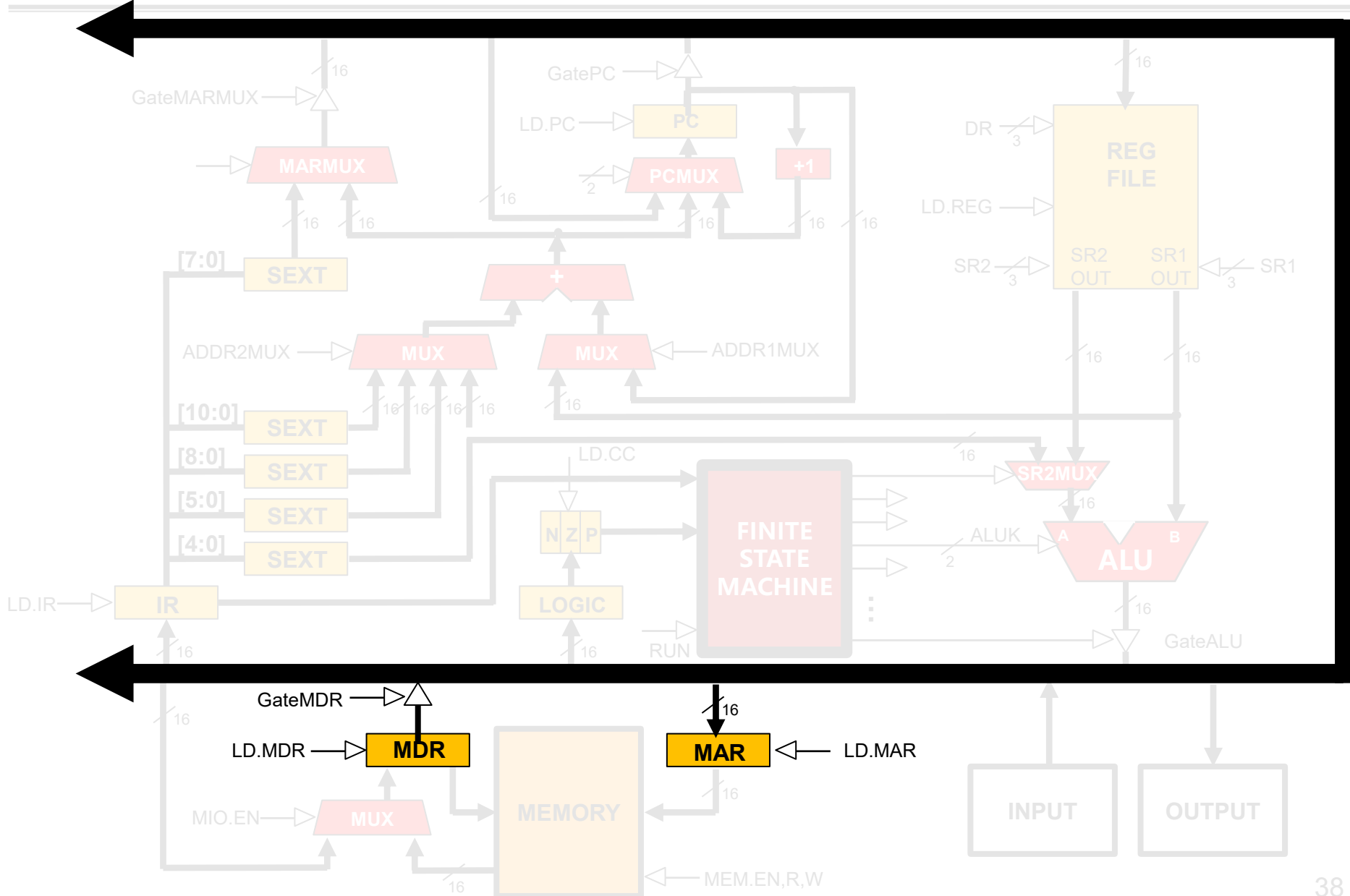
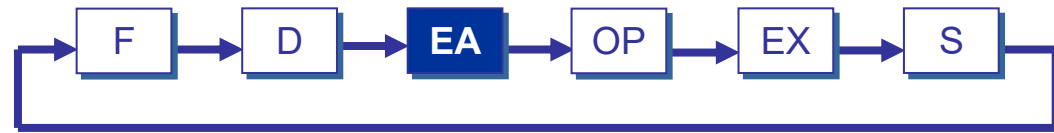
STI (Indirect)



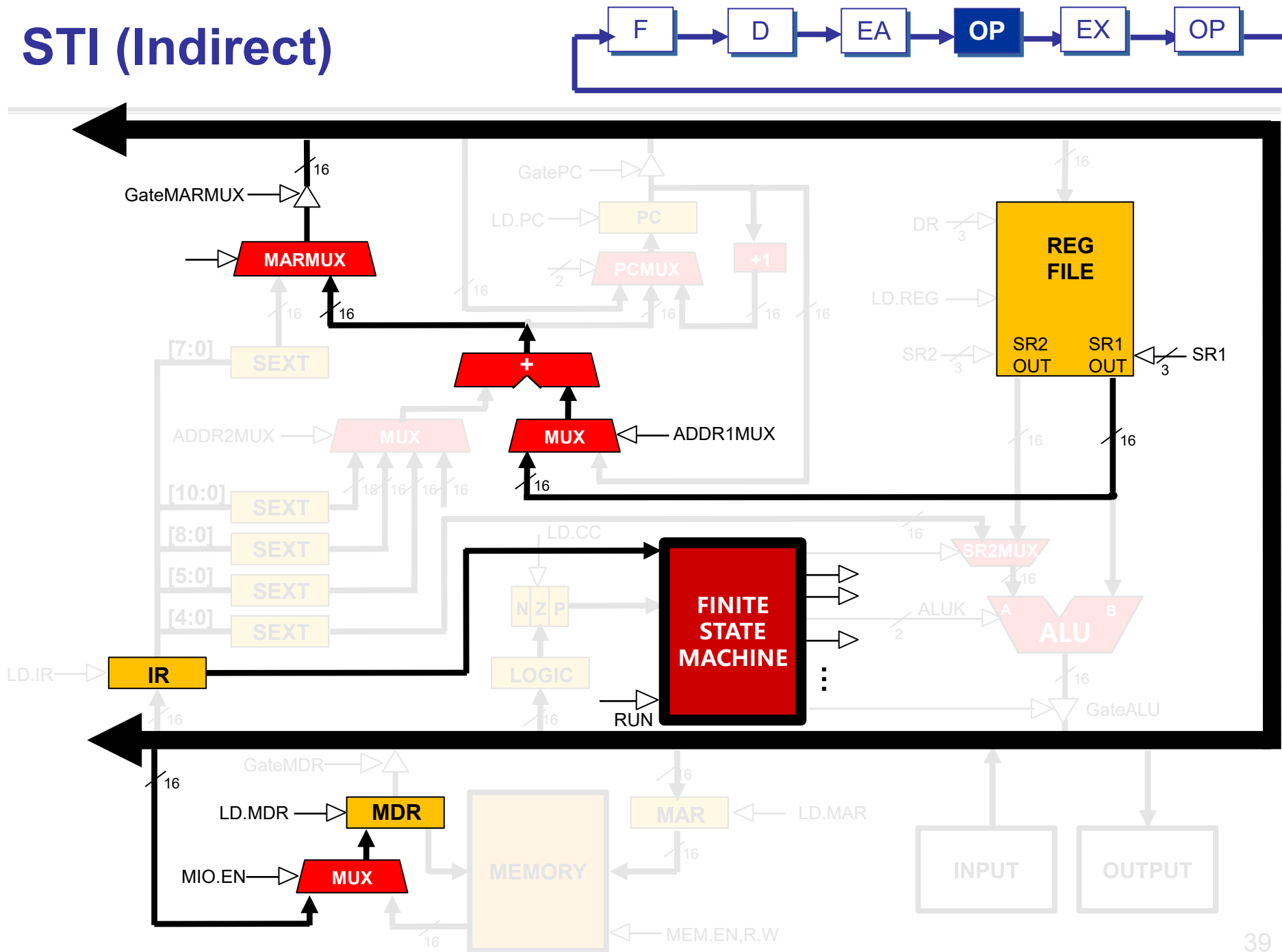
STI (Indirect)



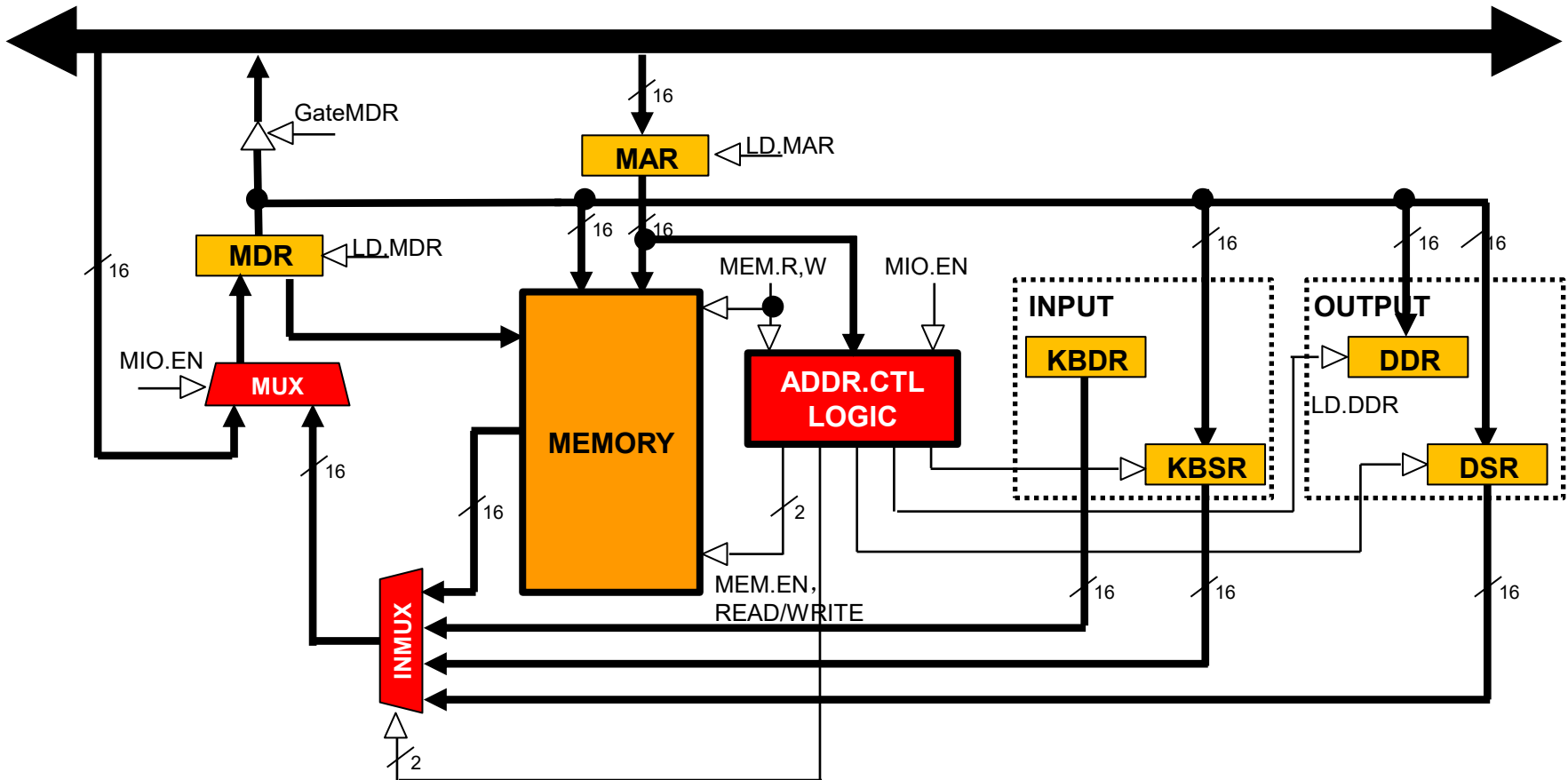
STI (Indirect)



STI (Indirect)



Memory-mapped I/O: STI R0, DDR ?



Keyboard Echo Routine

- Usually, input character is also printed to screen.
 - User gets feedback on character typed and knows its ok to type the next character.

```
POLL1    LDI    R0, KBSR
          BRzp   POLL1
          LDI    R0, KBDR
POLL2    LDI    R1, DSR
          BRzp   POLL2
          STI    R0, DDR

          . . .

KBSR     .FILL  xFE00
KBDR     .FILL  xFE02
DSR      .FILL  xFE04
DDR      .FILL  xFE06
```

